



République Algérienne Démocratique et Populaire



Ministère de l'Enseignement Supérieur et de la Recherche Scientifique

Université des Sciences et de la Technologie Houari Boumediene

Faculté d'Informatique

Département des Systèmes Informatiques

Mémoire de Licence

Spécialité :

Ingénierie des systèmes d'informations et des logiciels

Thème :

**Résolution du problème Bin Packing par
des méthodes approchées**

Présenté par :

BOUHADJAR Nada Abir

BENBOUCHAMA Fatima Zahra

Proposé par :

Prof. BOUKRA Abdelmadjid

Soutenu le: ... /.../...

Devant le jury composé de :

Président : **M. KADDOURI Lies**

Membre : **Mme. AMROUS Anissa Imene**

Projet n° : 04/2024.

Remerciements

Nous tenons tout d'abord à exprimer notre gratitude envers Allah, et le remercier pour nous avoir accordé la force et le courage de réaliser ce travail et le mener à bien.

En préambule de ce mémoire, nous souhaitons adresser nos remerciements les plus sincères aux personnes, qui par leur présence, leur soutien et leurs conseils ce projet a été possible.

Ces remerciements vont tout d'abord à notre encadreur Prof. BOUKRA qui s'est toujours montré à l'écoute et très disponible tout au long de la réalisation de ce projet, tant pour l'inspiration, l'aide et le temps qu'il a bien voulu nous consacrer et sans qui ce mémoire n'aurait jamais vu le jour.

Nous remercions également les membres du jury pour avoir accepté d'honorer par leur jugement ce travail.

Nous exprimons notre reconnaissance à toute personne ayant contribué de près ou de loin à la réalisation de ce projet.

Ce projet est désormais le résultat d'un travail durant trois années de formation à l'Université des Sciences et de la Technologie Houari Boumediene (USTHB), nous sommes fiers d'avoir fait partie de cette communauté.

Merci à tous et à toutes.

Dédicace

Je dédie ce mémoire à :

Ma mère

Pour sa patience et ses sacrifices. Ta prière et ta bénédiction m'ont été d'un grand secours tout au long de ma vie. Quoique je puisse dire et écrire, je ne pourrais exprimer ma grande affection et ma profonde reconnaissance. Puisse Dieu tout puissant, te préserver et t'accorder santé, longue vie et Bonheur

Mon père

Pour sa force, sa sagesse et son soutien constant. Papa, je vous remercie du fond du cœur pour votre présence rassurante et vos précieux conseils tout au long de mon parcours. Ce travail est dédié à votre amour et à votre soutien indéfectible.

Mes chers frères et sœurs

Pour leurs encouragements et leurs appuis permanents.

Mes grands-parents ainsi que toute ma famille, pour leur motivation et leur soutien tout au long de mon parcours universitaire.

Tous mes amis, pour leur soutien, leur aide et leurs témoignages d'amitié.

Bouhadjar Nada Abir

Dédicace

Je dédie ce mémoire à :

Ma mère

Pour ta tendresse infinie, ta force inébranlable et ton soutien indéfectible qui ont illuminé chaque étape de mon parcours. Je ne douterai jamais que tes prières pour moi et ta bénédiction ont été le plus grand secours tout au long de ma vie. Puisse Dieu tout puissant, te préserver toi et mon père et t'accorder santé, longue vie et bonheur.

Mon père

Pour ta sagesse, ta confiance en moi ainsi que ton soutien constant tout au long de ce voyage académique. Je ne te remercierai jamais assez pour ta présence rassurante et tes précieux conseils tout au long de cette aventure intellectuelle. Ce travail est un hommage humble à ton influence inestimable.

Mes chers frères

Pour leurs encouragements et leurs appuis permanents.

Mes grands-parents ainsi que toute ma famille, pour leur motivation et leur soutien tout au long de mon parcours universitaire.

Tous mes amis, ayants cru en moi, pour leur soutien, leur aide et leurs témoignages d'amitié.

Benbouchama Fatima Zahra

Table des matières

Introduction générale et définition du problème du <i>Bin packing</i>	1
Chapitre 1 : Etat de l'art	5
1.1. Introduction :	5
1.2. Problématique :	5
1.3. Champs d'application du problème du <i>Bin packing</i> :	5
1.3.1. Gestion des Ressources Informatiques :	5
1.3.2. Gestion des Stocks :	6
1.3.3. Découpe de Matériaux :	6
1.4. Définition des différentes variantes du problème du <i>Bin packing</i> :	6
1.4.1. <i>Bin packing</i> avec des contraintes de capacité différentes :	6
1.4.2. <i>Bin packing</i> avec coûts :	7
1.4.3. <i>Bin packing</i> avec priorités :	7
1.4.4. <i>Bin packing</i> avec contraintes temporelles :	7
1.4.5. <i>Bin packing</i> avec des objets fragiles :	7
1.4.6. <i>Bin packing</i> avec des objets dépendants :	7
1.5. Formulation mathématique du problème :	7
1.6. Quelques travaux sur le problème du <i>Bin packing</i> :	8
1.7. Conclusion :	9
Chapitre 2 : Notion d'optimisation	11
2.1. Introduction :	11
2.2. Méthodes de résolution des problèmes d'optimisation :	12
2.2.1. Méthodes exactes :	12
2.2.2. Méthodes approchées :	13
2.3. Description des trois algorithmes à utiliser :	15
2.3.1. Algorithmes génétiques :	15
2.3.2. Algorithme des colonies de fourmis :	16
2.3.3. Algorithmes d'Approximation :	17
2.4. Conclusion	19
Chapitre 3 : Résolution du problème (<i>Bin packing</i>) par deux méthodes approchées	21
3.1. Introduction :	21
3.2. Codage de la solution :	21

3.3.	Description des deux approches :	22
3.3.1.	Algorithme génétique :	22
3.3.2.	Algorithme de colonie de fourmis :	25
3.3.	Conclusion :	27
4.1.	Introduction :	29
4.2.	Description des benchmarks :	29
4.2.1.	Les benchmarks de la littérature :	29
4.2.2.	Les benchmarks personnalisés :	30
4.3.	Description de l'interface développée :	30
4.3.1.	Caractéristiques de la machine utilisée (Hardware) :	30
4.3.2.	Langages de programmations et outils utilisés :	30
4.4.	Expérimentation et analyse :	32
4.4.1.	Résultat expérimentaux :	32
4.4.2.	Discussion des résultats :	33
4.5.	Conclusion :	36
	Conclusion générale	37
	Bibliographie	39
	Annexe	43

Liste des figures

Figure 1 : Classification des méthodes de résolution des problèmes d'optimisation.....	12
Figure 2 : L'optimisation par essaims de particules.....	15
Figure 3 : Processus de l'algorithme génétique	16
Figure 4 : Exemple du déroulement de l'algorithme du Best-fit.....	19
Figure 5 : illustration du processus de l'algorithme génétique pour un nombre d'itération fini	22
Figure 6 : Tableau des caractéristiques de la machine utilisée.....	30
Figure 7 : À droite le logo de l'environnement Pycharm et à gauche le logo de python.....	30
Figure 8 : À gauche de logo d'HTML et à droite le logo du CSS	31
Figure 9 : À gauche le logo de JavaScript et a droite le logo du Framework Flask.....	31
Figure 10 : Logo de l'environnement de développement de l'application « VISUAL CODE STUDIO »	32
Figure 11 : Tableau comparatifs des résultats obtenus des deux algorithmes et des résultats des benchmarks théoriques cités	33
Figure 12 : Graphe comparatifs illustrant le nombre de bacs utilisés pour chaque algorithme en fonction du nombre d'objets.....	34
Figure 13 : Graphe comparatif illustrant le temps de convergence des deux algorithmes en fonction du nombre d'objets à classer.....	35
Figure 14 : Page d'accueil de l'application.....	43
Figure 15 : Formulaire des données d'entrée	43
Figure 16 : Formulaire des différentes données nécessaires pour l'exécution d'un algorithme	44
Figure 17 : Table des benchmarks théoriques enregistrés dans la base de données.	44
Figure 18 : Table des benchmarks personnalisés	45
Figure 19 : Une instance de la table des benchmarks personnalisés	45
Figure 20 : Formulaire du benchmark sélectionné.....	45
Figure 21 : Section du choix entre les deux métaheuristiques	46
Figure 22 : Les paramètres de l'algorithme génétique	46
Figure 23 : Les paramètres de l'algorithme de colonies de fourmis	46
Figure 24 : Les résultats de l'algorithme	47
Figure 25 : Boutons générants du graphe.....	47

Introduction générale et définition du problème du *Bin packing*

Au fil des siècles, parmi les concepts les plus explorés par de multiples chercheurs, un les a toujours intrigués : la recherche de la meilleure solution parmi un ensemble fini de solutions possibles. Généralement, cette recherche se fait en respectant certaines contraintes tel que le temps limité, l'espace mémoire limité ou encore les ressources limitées. Le problème du *Bin packing*, ou plus généralement le problème de rangements et de découpes, relève de ce concept.

Imaginons, maintenant, un ensemble de sacs identiques et un ensemble d'objets dont le poids est connu. Sachant que les sacs ne peuvent supporter qu'un poids maximum, combien de sacs nous faudra-t-il, au minimum, pour y ranger l'ensemble des objets considérés [1] ?

Ce défi d'optimisation est l'exemple parfait illustrant le sens du problème de « *Bin packing* » signifiant : le problème du conditionnement dans des boîtes, ou en langage simple « problème d'emballage dans des bacs ». Bien que paraissant simple en apparence, ce défi est en réalité complexe.

Cependant, ce dernier n'est pas exclusivement lié aux problèmes mathématiques, mais il est aussi associé à de nombreuses applications réelles (service de biens ou problèmes informatiques). Nous pouvons citer sans être exclusif, le placement optimal des articles dans des journaux, la gestion des stocks dans une entreprise proposant des biens matériels, l'industrie de découpe de tissu, la logistique, l'affectation des tâches aux processeurs ...etc.

Bien que ces domaines peuvent nous sembler très éloignés les uns des autres, mais ils bénéficient tous d'un seul moyen de résolution : le *Bin packing*.

La littérature sur le problème de *Bin packing* est riche et les algorithmes et approches de résolution sont très diverses. Ces derniers ont été étudiés et traités par la communauté de la recherche opérationnelle et de l'optimisation combinatoire les classant parmi les problèmes NP-complets ou NP-difficiles [2], ce qui signifie qu'il est difficile de trouver une solution optimale en un temps polynomial [3]. Néanmoins, différentes techniques et méthodes heuristiques et algorithmiques sont souvent utilisées pour trouver des solutions satisfaisantes dans un délai raisonnable, même si elles ne garantissent pas l'optimalité totale [4].

Dans ce contexte, l'objectif de notre travail est d'étudier les différentes approches permettant de résoudre notre problématique, le problème du *Bin packing*, et d'en choisir

quelques-unes pour comparer leurs performances et déterminer la quelle est la plus appropriée pour atteindre notre objectif.

Dans le premier chapitre, nous avons présenté brièvement notre problématique ainsi que ses différentes contraintes, et nous avons exploré les divers domaines d'application de celle-ci. Enfin, nous avons présenté quelques travaux réalisés ayant un objectif similaire au nôtre.

Le chapitre suivant a permis de définir et d'introduire le terme « optimisation » et d'aborder les différentes classes de méthodes d'optimisation en mentionnant les avantages et les inconvénients de chacune.

Dans le troisième chapitre, après avoir exploré les différentes méthodes d'optimisation abordées dans le chapitre précédent, nous avons choisi deux métaheuristiques : la première est l'approche génétique combinée avec l'heuristique du Best-fit, et la deuxième est l'approche de la colonie de fourmis. Nous avons également expliqué le processus de fonctionnement de chaque approche.

En définitive, dans le dernier chapitre, nous avons d'abord présenté brièvement l'interface que nous avons développée. Ensuite, afin de comparer les performances des deux approches choisies, nous avons effectué plusieurs exécutions sur des instances précédemment utilisées par d'autres chercheurs, ce qui nous a permis de tirer une conclusion sur notre travail.

Chapitre 1 : Etat de l'art

Chapitre 1 : Etat de l'art

1.1. Introduction :

Je veux déménager bientôt, et je me demande si le seul camion que j'ai loué suffira pour transporter tous mes cartons ? Eh bien, je me trouve confronté à l'un des problèmes fondamentaux d'optimisation, à savoir le "problème du *Bin packing*".

Le problème *Bin packing*, fondamental dans l'optimisation combinatoire, se présente comme un défi d'allocation efficace d'objets dans des conteneurs afin de minimiser les espaces inutilisés. Son impact s'étend à des domaines variés qui vont être cités avec plus de détails dans ce qui va suivre.

Cette problématique englobe diverses variantes représentant des contraintes de capacité aux priorités d'objets, pouvant ainsi varier la formulation mathématique du problème qui repose sur la minimisation du nombre de bacs utilisés tout en respectant les contraintes spécifiées en entrée.

1.2. Problématique :

Supposons qu'on dispose d'un certain nombre d'objets, et que nous voulons classer ces derniers dans des bacs de capacité connue, combien de bacs faudra-t-il au minimum pour atteindre notre objectif tout en respectant les contraintes imposées ? Ce problème est connu sous le nom du «*Bin packing problem*», qui est une problématique majeure ayant des implications pratiques importantes dans l'amélioration de l'efficacité opérationnelle et la gestion efficace des ressources.

1.3. Champs d'application du problème du *Bin packing* :

Le problème du *Bin packing*, également connu sous le nom de problème d'emballage de bin, trouve des applications dans divers domaines. Voici quelques champs d'application significatifs de ce problème :

1.3.1. Gestion des Ressources Informatiques :

La gestion des ressources informatiques par *Bin packing* consiste à optimiser l'allocation des ressources, telles que l'espace de stockage ou la capacité des serveurs, dans des

environnements informatiques. Ce problème trouve des applications cruciales dans les centres de données et les infrastructures Cloud, où l'efficacité opérationnelle et l'utilisation des ressources sont des impératifs.

Le *Bin packing* informatique vise à minimiser les espaces inutilisés dans les serveurs ou les dispositifs de stockage, réduisant ainsi les coûts associés à l'énergie, au refroidissement et à l'expansion physique des infrastructures.

1.3.2. Gestion des Stocks :

La gestion des stocks par *Bin packing* consiste à optimiser l'allocation d'articles dans des conteneurs, ou "bacs", de manière à minimiser l'espace inutilisé. Ce problème trouve des applications pratiques dans la logistique et la gestion des entrepôts, visant à maximiser l'efficacité de l'espace de stockage.

En réalité, l'algorithme de *Bin packing* assigne les articles à des bacs en fonction de leurs dimensions et de leurs exigences de stockage, cherchant à minimiser le nombre total de bacs utilisés. Cette approche permet d'optimiser l'utilisation de l'espace de stockage, réduisant ainsi les coûts associés au stockage et à l'espace physique nécessaire.

1.3.3. Découpe de Matériaux :

La découpe de matériaux par *Bin packing* consiste à optimiser le découpage de matériaux en morceaux de tailles spécifiques, de manière à minimiser les chutes et à utiliser efficacement les ressources. Cette approche trouve des applications dans des industries telles que la fabrication, la menuiserie ou la production de textiles.

L'algorithme de *Bin packing* attribue les dimensions des pièces à découper aux différentes sections du matériau de manière à minimiser les pertes. L'objectif est d'organiser la découpe de manière à remplir au mieux la feuille de matériau, réduisant ainsi le gaspillage et optimisant l'utilisation des ressources.

1.4. Définition des différentes variantes du problème du *Bin packing* :

1.4.1. *Bin packing* avec des contraintes de capacité différentes :

L'objectif est de minimiser le nombre total de bacs utilisés pour placer tous les objets, en tenant compte des contraintes de capacité spécifiques de chaque bac. Cela signifie que lorsqu'on place un objet dans un bac, il faut prendre en compte non seulement la taille de

l'objet, mais aussi la capacité disponible dans le bac. Ceci ajoute une dimension supplémentaire à la complexité du problème.

1.4.2. *Bin packing* avec coûts :

Dans ce type de problème, chaque objet a un coût associé, et l'objectif n'est pas seulement de minimiser le nombre de bacs utilisés, mais aussi de minimiser le coût total des bacs utilisés.

1.4.3. *Bin packing* avec priorités :

Ce type de problème ajoute une dimension supplémentaire au problème classique du *Bin packing* en attribuant des priorités différentes aux objets à placer dans les bacs. Chaque objet est associé à une priorité, et l'objectif est de minimiser le nombre total de bacs utilisés tout en tenant compte de ces priorités.

1.4.4. *Bin packing* avec contraintes temporelles :

Le problème du *Bin packing* avec contraintes temporelles est une variante du problème classique du *Bin packing* qui introduit des contraintes liées au temps dans le processus de placement des objets dans les bacs. Concrètement, cela signifie que chaque objet a une date limite ou une contrainte temporelle avant laquelle il doit être placé dans un bac.

1.4.5. *Bin packing* avec des objets fragiles :

Dans ce type de problème, nous prenons en compte la fragilité des objets à placer dans les bacs. Cependant, cette dernière suppose que certains objets ne peuvent pas être placés dans le même bac avec d'autres objets en raison de leur fragilité.

1.4.6. *Bin packing* avec des objets dépendants :

Dans cette variante, certains objets sont interdépendants, ce qui signifie qu'ils doivent être placés dans le même bac.

1.5. Formulation mathématique du problème :

Imaginons que nous possédons n objets, où chaque objet « i » a une taille « s_i » (pour $i=1,2,\dots, n$), ajoutons aussi que chaque bac a une capacité « C ».

« x_{ij} » est considérée comme variable binaire indiquant si l'objet « i » est placé dans le bac « j » tel que :

$x_{ij}=1$ si l'objet i est placé dans le bac j

$x_{ij}=0$ sinon

y_j est une variable binaire indiquant si le bac j est utilisé tel que :

$y_j = 1$ si le bac j est utilisé

$y_j = 0$ sinon

La fonction objectif du problème permettra de minimiser la formule suivante :
 $\sum_{j=1}^N y_j$ (où N est le nombre total de bacs)

Contraintes :

a. Chaque objet doit être placé exactement dans un bac :

$$\sum_{j=1}^N x_{ij} = 1 \quad \text{Pour } i=1,2,\dots, n$$

b. La capacité de chaque bac ne doit pas être dépassée :

$$\sum_{i=1}^n s_i \cdot x_{ij} \leq C \quad \text{Pour } j=1,2,\dots, N$$

c. Les variables de décision sont binaires :

$$x_{ij} \in \{0,1\} \quad \text{pour } i=1,2,\dots, n \text{ et } j=1,2,\dots, N$$

d. y_j est binaire :

$$y_j \in \{0,1\} \quad \text{pour } j=1,2,\dots, N$$

1.6. Quelques travaux sur le problème du *Bin packing* :

De nombreux travaux ont été réalisés pour proposer des approches et des algorithmes pour résoudre ce problème. Nous pouvons citer les suivants :

- En 2018, une étude a appliqué PSO (algorithme des essaims particulaires) pour résoudre le problème du rangement de produits dans une entreprise hospitalière, en tenant compte des priorités et des incompatibilités entre les éléments et les conteneurs .Ces recherches montrent que PSO peut être utile pour résoudre des problèmes de *Bin packing* complexes et variés. Les performances de ces méthodes ont souvent été comparées favorablement aux autres algorithmes de résolution, comme le recuit simulé (SA) [5].
- Une étude a été menée en 2020 pour résoudre le problème de *Bin packing* à taille de bacs multiples en utilisant l'algorithme de recuit simulé (simulated annealing). L'étude a comparé différentes stratégies de refroidissement empiriquement et a donné une solution pour un problème de 100 éléments qui est probablement la meilleure connue à ce jour [6].

- Une étude a été publiée concernant une méthode hybride d'algorithmes évolutifs (HEA) pour résoudre le problème de *Bin packing* à une dimension (BPP) dans le domaine des opérations centrales européennes. Cette méthode utilise une représentation individuelle basée sur les bacs et emploie deux nouveaux opérateurs de mutation basés sur une matrice fréquence des couples d'éléments (RFPM). L'HEA a été testé sur des instances bien connues du BPP et s'est avéré efficace pour le problème difficile du test *set hard28* et dans la plupart des problèmes de test, il a pu atteindre l'optimum [7].
- Une autre étude a proposé un algorithme génétique pour résoudre le problème de *Bin packing* de conteneurs dans les ports. L'objectif de cette étude était de déterminer un arrangement valide des conteneurs dans le port, en utilisant un algorithme génétique pour optimiser la disposition des conteneurs dans les navires [8].

1.7. Conclusion :

Ces dernières années, le problème du *Bin packing* a été au centre de l'attention dans le domaine de l'optimisation combinatoire. Il représente un défi crucial pour l'allocation efficace d'objets dans des conteneurs afin de minimiser les espaces inutilisés tout en optimisant l'utilisation des ressources et en réduisant les coûts.

Diverses méthodes ont été mises en œuvre pour résoudre ce problème, chacune apportant sa propre contribution à l'amélioration des performances des résultats obtenus.

En définitive, l'évolution des techniques de résolution de ce problème n'a jamais connue de pause. Des approches récentes, plus performantes, telles que les "hyper-heuristiques" ou "méta-heuristiques", ont émergé pour combiner intelligemment différentes heuristiques déjà employées dans des contextes spécifiques. Ces méthodes visent à optimiser l'application des heuristiques de manière efficace, cherchant ainsi à améliorer significativement les performances dans la résolution de problèmes.

Chapitre 2 : Notion d'optimisation

Chapitre 2 : Notion d'optimisation

2.1. Introduction :

Dans de nombreuses applications dans le monde réel, telles que la logistique, la planification des horaires, la conception de réseaux, etc., il y a toujours eu le besoin de trouver des configurations optimales à partir d'un ensemble discret d'objets. Ceci est connu en tant que problème d'optimisation combinatoire.

L'optimisation est un concept fondamental englobant le processus d'amélioration des performances ou de maximisation d'un système, d'un processus ou d'un résultat visant à atteindre les meilleurs résultats possibles, dans un contexte spécifique et dans des conditions précises. En d'autres termes, c'est rendre quelque chose aussi efficace, performante ou optimale que possible. Cela peut impliquer divers aspects telles la minimisation des coûts, la recherche de la meilleure configuration, ...etc.

Lorsque nous abordons un problème d'optimisation, on a généralement un objectif spécifique que l'on souhaite atteindre. La fonction mathématique mesurant l'efficacité de la solution potentielle et sa performance est connue sous le nom de la "fonction objectif". Celle-ci évalue la qualité de la solution en fonction des valeurs des variables d'optimisation qui peuvent être ajustées.

Cependant, il existe des situations où la formulation d'une fonction objectif [9] analytique, précise et explicite peut être difficile ou même impossible.

Dans certains domaines, notamment en intelligence artificielle (IA), les solutions peuvent être symboliques ou non numériques, en raison de la nature intrinsèque des problèmes rencontrés, rendant difficile l'expression d'un objectif sous forme d'une fonction mathématique continu.

Afin de surmonter ces défis, l'introduction de nouvelles méthodes d'optimisation devient cruciale. Ces méthodes, telles que la descente du gradient, les algorithmes génétiques, la recherche tabou, ou encore l'optimisation bayésienne offrent des approches flexibles, c'est-à-dire des méthodes qui peuvent être adaptées et appliquées de manière versatile à différents types de problèmes pour explorer efficacement l'espace des solutions, même lorsque la formulation d'une fonction objectif explicite est complexe.

2.2. Méthodes de résolution des problèmes d'optimisation :

Il est important de choisir la méthode d'optimisation appropriée en fonction de la nature du problème que l'on essaye de résoudre, de la complexité de l'espace des solutions et des contraintes spécifiques associées au problème. Certains problèmes peuvent bénéficier de l'utilisation combinée de plusieurs méthodes d'optimisation.

Il existe de nombreuses méthodes (illustrées dans la figure ci-dessous), chacune adaptée à des types spécifiques de problèmes.

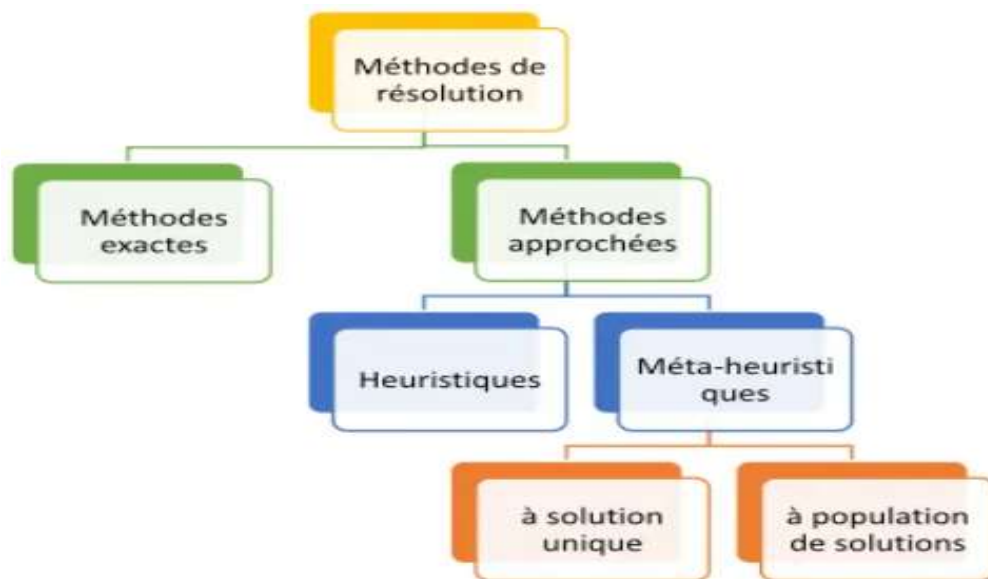


Figure 1 : Classification des méthodes de résolution des problèmes d'optimisation

Voici une brève présentation de quelques méthodes d'optimisation :

2.2.1. Méthodes exactes :

Les méthodes exactes (appelées aussi complètes) produisent une solution optimale pour une instance donnée de problème d'optimisation c'est-à-dire la meilleure solution possible selon un critère spécifique, tout en garantissant l'exhaustivité et la garantie mathématique de la solution optimale. Elles reposent généralement sur la recherche arborescente et sur l'énumération partielle de l'espace de solutions.

On trouve deux grandes classes d'algorithmes exacts en optimisation combinatoire : les algorithmes de type séparation-évaluation (SE).

Les algorithmes SE construisent des solutions partielles au problème puis tentent de les compléter.

Les méthodes de coupe construisent une suite de solutions non optimales, que l'algorithme exclut de l'ensemble des solutions recherchées jusqu'à trouver une solution optimale.

Bien que ces méthodes garantissent une solution optimale en sortie, mais elles peuvent présenter des inconvénients majeurs. Elles sont souvent confrontées à une complexité temporelle élevée, en particulier pour des problèmes NP-complets, où la complexité temporelle des algorithmes exacts augmente de manière exponentielle avec la taille de l'instance, ce qui signifie que le temps de calcul peut devenir rapidement exorbitant pour des instances de taille significative, rendant leur application impraticable.

C'est l'une des raisons pour lesquelles d'autres approches, telles que les méthodes heuristiques, sont souvent préférées pour obtenir des solutions acceptables dans des délais plus raisonnables [10].

2.2.2. Méthodes approchées :

2.2.2.1. Les Heuristiques :

Les heuristiques représentent des approches algorithmiques qui visent à fournir rapidement des solutions à divers problèmes décisionnels. En effet, l'utilisation de méthodes de programmation mathématique peut parfois entraîner une complexité mathématique considérable, entraînant des temps de calcul prolongés. En pratique, pour répondre aux impératifs de délais dans la résolution de problèmes décisionnels, il est courant de sacrifier partiellement la qualité mathématique de la solution afin d'accélérer le processus de résolution. En d'autres termes, on peut accepter une solution qui n'est pas nécessairement la meilleure d'un point de vue mathématique, mais qui permet de trouver rapidement une réponse satisfaisante.

Ainsi, une heuristique est une stratégie permettant de se déplacer intelligemment dans l'espace des solutions, afin d'obtenir une solution approchée, la meilleure possible, dans un délai de temps raisonnable.

Les heuristiques sont dépendantes du problème à résoudre, principalement dans le choix du voisinage (le déplacement dans l'espace des solutions).

Deux types d'heuristiques sont principalement utilisées : les heuristiques de construction (par exemple les méthodes gloutonnes), qui construisent itérativement une solution, et les heuristiques de descente, qui à partir d'une solution donnée cherchent un optimum local.

Des heuristiques plus poussées ont été mises au point et ont donné naissance à une nouvelle famille d'algorithmes (qui sera détaillé plus dans le prochain point) : les méta-heuristiques [11].

2.2.2.2. Les Métaheuristiques :

Le terme métaheuristique est composé de « méta », provenant du grec ancien, signifiant "au-delà", traduisible par "à un plus haut niveau", et d'heuristique évoqué plus haut." Ce terme a été mentionné pour la première fois lors de la conception de la recherche tabou. Les métaheuristiques sont généralement des algorithmes stochastiques [12] itératifs, qui progressent vers un optimum global tout en utilisant des stratégies indépendantes de la structure du problème auquel elles sont appliquées.

L'approche vise à éviter les optimums locaux par une stratégie d'interdiction (ou, plus généralement, de pénalisation) de certains mouvements, ce qui signifie que l'enjeu pour les méta-heuristiques est de ne pas rester bloquées dans un optimum local.

Cela peut être évité en veillant à respecter l'équilibre entre exploration (ou diversification) et exploitation (ou intensification).

L'exploration décrit la capacité de l'algorithme à parcourir l'ensemble de l'espace de recherche S pour cibler des régions prometteuses. Tandis que l'exploitation est la capacité d'explorer précisément une région de l'espace de recherche.

Ces méthodes utilisent cependant un haut niveau d'abstraction, ce qui leur permet d'être adaptées à une large gamme de problèmes différents [13] [14].

2.2.2.2.1. Exemples de métaheuristiques :

2.2.2.2.1.1. Optimisation par Essaim de Particules (PSO) :

Dans cette métaheuristique, une population de particules est initialement positionnée aléatoirement dans l'espace de recherche.

Chaque particule représente une solution potentielle au problème d'optimisation. Les particules ajustent leurs positions et leurs vitesses au fil des itérations en fonction de leur propre expérience et de celle du groupe. Chaque particule maintient une mémoire de sa meilleure position individuelle (pbest), et le groupe a une mémoire collective de la meilleure position globale (gbest). Les particules se déplacent vers ces positions optimales en utilisant des formules qui intègrent des facteurs d'inertie, des composantes personnelles et sociales. Ce processus d'ajustement des positions et des vitesses est répété jusqu'à ce

qu'un critère d'arrêt soit atteint, comme un nombre fixe d'itérations ou l'obtention d'une solution suffisamment performante.

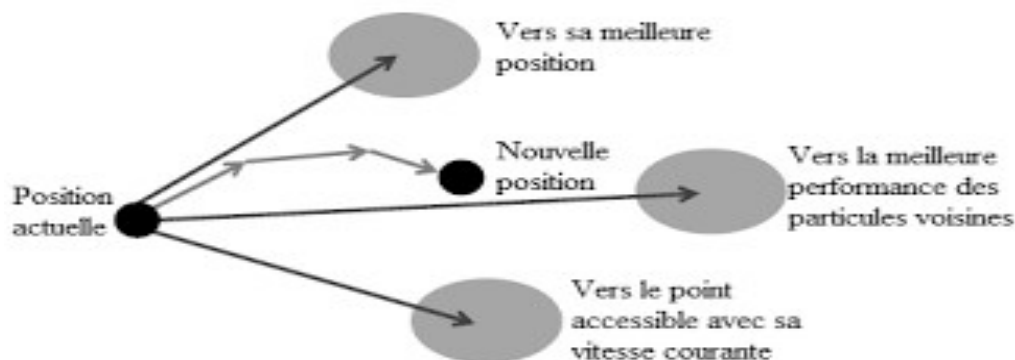


Figure 2 : L'optimisation par essaims de particules

L'optimisation par PSO a été appliquée pour la 1^{ère} fois dans le domaine des systèmes d'alimentation pour minimiser la perte de puissance active d'un réseau électrique. Ce fut la première application du PSO dans le domaine de l'énergie électrique [15].

2.3. Description des trois algorithmes à utiliser :

2.3.1. Algorithmes génétiques :

Les Algorithmes Evolutionnaires (AE) faisant partie des méthodes méta-heuristiques et inspiré par les travaux du biologiste Charles Darwin, sont des techniques d'optimisation évolutionnaire basées sur les principes d'utilisation des opérations telles que la sélection naturelle, la recombinaison et la mutation pour évoluer vers des solutions optimales[16][17]. Dans cette approche, nous parlerons donc d'individus (solutions potentielles), de population, de gènes (variables), de chromosomes, de parents, de descendants, de reproduction, de croisement, de mutations...etc. Et nous nous appuierons constamment sur des analogies avec les phénomènes biologiques.

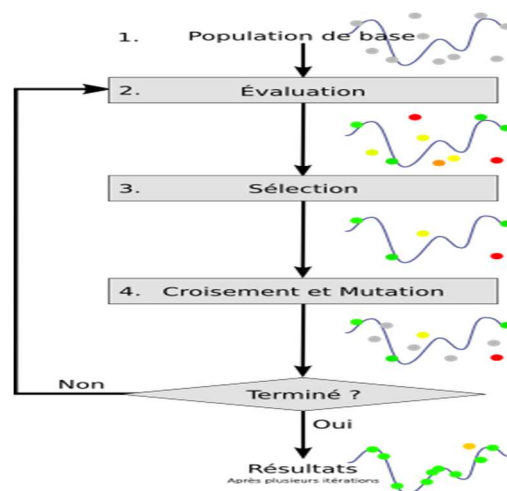


Figure 3 : Processus de l'algorithme génétique

Initialement, comme l'illustre la figure précédente une population d'individus est générée, chaque individu représentant une solution potentielle au problème (positionnement des objets dans les bacs).

Ces solutions sont ensuite évaluées en fonction d'une fonction objectif que nous allons appeler « fonction de *fitness* ». Le fonctionnement de cette dernière consiste à renvoyer une valeur représentant la qualité de chaque solution de la population initiale.

La sélection des individus pour la reproduction est effectuée ensuite en fonction de leur performance, imitant ainsi le principe de survie des plus adaptés (individus ayant une valeur de *fitness* élevée).

La reproduction se fait par croisement, où les caractéristiques des individus sélectionnés sont combinées pour créer de nouveaux individus, simulant ainsi le processus génétique.

Des mutations peuvent également être appliquées pour introduire une diversité génétique.

Ce cycle d'évaluation, de sélection, de reproduction et de remplacement est répété jusqu'à ce qu'une condition d'arrêt prédéfinie soit atteinte. Pour notre cas nous avons choisi d'effectuer un nombre précis d'itération.

2.3.2. Algorithme des colonies de fourmis :

Cette métaheuristique inspirée elle aussi de la nature, a été introduite en 1991 par Colorni, Dorigo et Maniezzo pour résoudre le problème du Voyageur de commerce. Elle s'est popularisée, puis a été l'objet d'améliorations dès 1995 et a été appliquée avec succès à d'autres problèmes d'optimisation combinatoire dès 1994 [18].

L'idée originale provient de l'observation de l'exploitation des ressources alimentaires chez les fourmis. En effet, celles-ci, bien qu'ayant individuellement des capacités cognitives

limitées, sont capables collectivement de trouver le chemin le plus court entre une source de nourriture et leur nid.

Les fourmis sont souvent chargées de trouver de la nourriture pour nourrir la colonie. Elles parcourent leur environnement à la recherche de sources de nourriture, telles que des morceaux de nourriture ou des plantes sucrées.

Une fois qu'une fourmi trouve de la nourriture, elle retourne à la colonie en suivant un chemin plus ou moins direct. Pendant son voyage, elle dépose des phéromones sur le sol. Les autres fourmis de la colonie détectent ces phéromones et les suivent pour trouver la source de nourriture. Ce qui rend en quelque sorte la phéromone déposée par une fourmi proportionnelle à la qualité de la solution qu'elle ont découvert.

Les chemins qui mènent à la source de nourriture sont renforcés par les phéromones laissées par les fourmis qui les empruntent. Les chemins les plus courts et les plus efficaces sont renforcés par des concentrations plus élevées de phéromones, ce qui attire davantage de fourmis.

Les fourmis exploitent les ressources de manière collective. Une fois qu'une source de nourriture est trouvée, un grand nombre de fourmis sont mobilisées pour collecter la nourriture et la ramener à la colonie.

Enfin, le comportement des fourmis dans une colonie est caractérisé par une combinaison d'exploration, de communication, d'exploitation des ressources, et d'adaptation aux changements environnementaux. Ces caractéristiques sont souvent reproduites dans les algorithmes de colonies de fourmis pour résoudre des problèmes d'optimisation telle que le *Bin packing problem*.

Après discussion de la façon dont peut-on transformer le comportement des fourmis lors de la recherche de leurs nourritures en un algorithme pouvant résoudre notre problème nous avons pu aboutir à une solution qui a été expliquée en détails dans le chapitre qui suit.

2.3.3. Algorithmes d'Approximation :

Les algorithmes d'approximation pour le problème du *Bin packing* généralement considérés comme des heuristiques, cherchent à fournir des solutions proches de l'optimalité en temps polynomial. On peut fréquemment retrouver ces algorithmes dans le processus d'allocation d'espace mémoire aux différents programmes. Bien que ses algorithmes ne garantissent pas une solution optimale dans tous les cas, ils sont souvent utilisés dans la pratique en raison de leur efficacité [19].

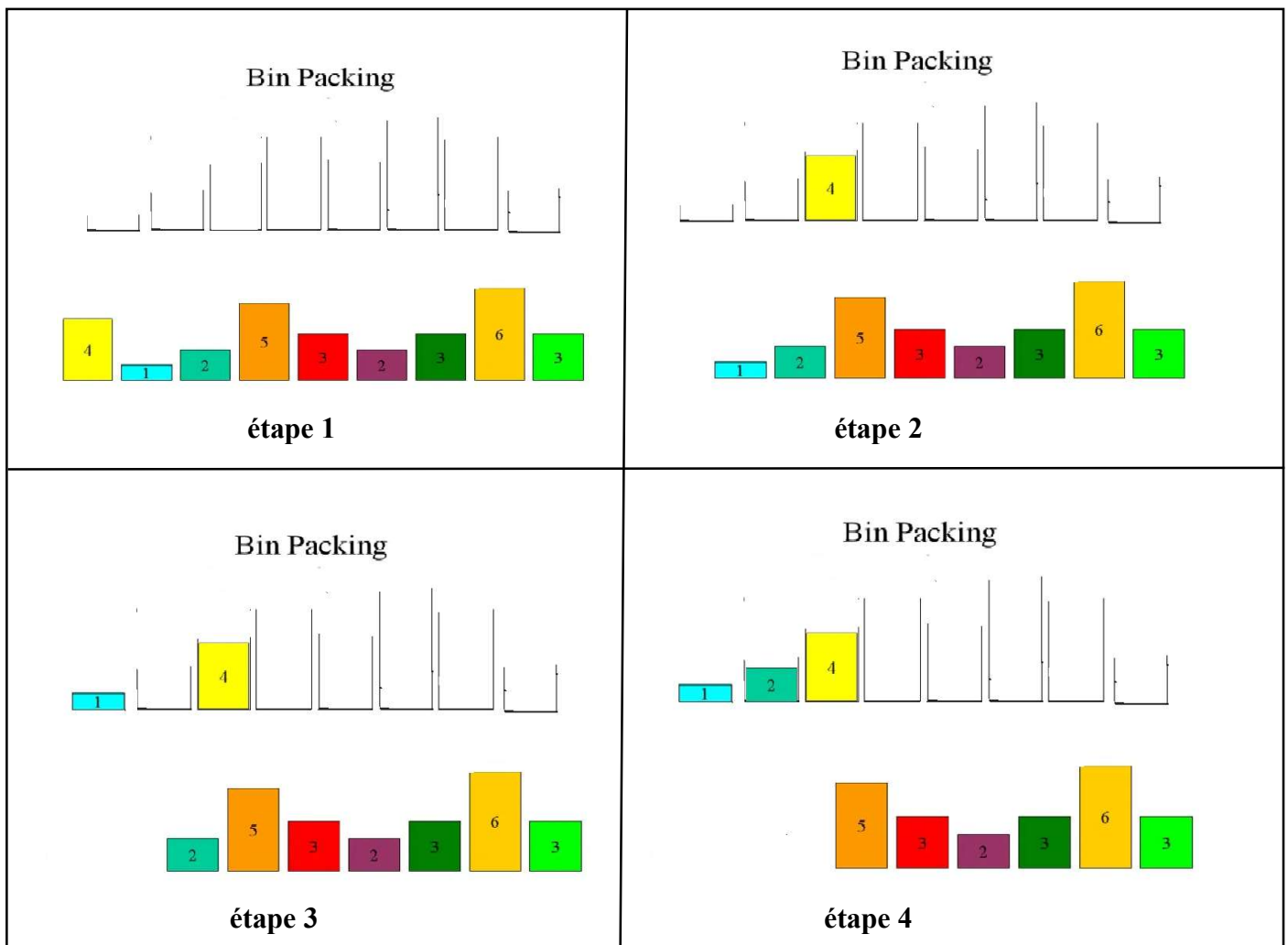
Dans le cadre de notre projet, nous aborderons l'un de ces algorithmes, qui est l'algorithme Best Fit [20].

2.3.3.1. Best-Fit (Meilleur Ajustement) :

L'idée principale de l'algorithme Best Fit est de placer chaque objet dans le bac qui a le moins d'espace restant après l'ajout de cet objet. Voici comment fonctionne l'algorithme Best Fit :

1. On commence par créer une liste vide de bacs.
2. Pour chaque objet, on recherche le bac de capacité la plus proche de la taille de l'objet, sans la dépasser.
3. Si un bac de capacité adaptée est trouvé, on y alloue l'objet.
4. Si un bac est rempli, on le retire de la liste des bacs disponibles.

Voici une figure illustrant un exemple simple du déroulement de l'algorithme du Best-fit :



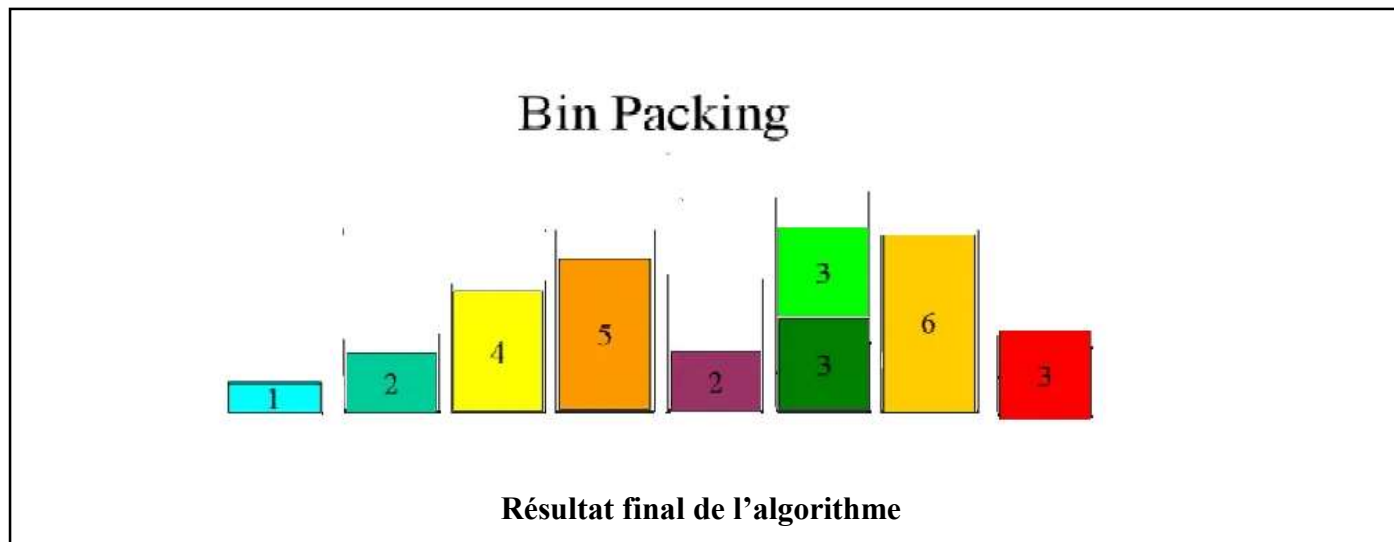


Figure 4 : Exemple du déroulement de l'algorithme du Best-fit

2.4. Conclusion :

En résumé, ce chapitre a mis en lumière l'importance cruciale de l'optimisation dans la résolution de problèmes complexes, en particulier en intelligence artificielle. De la conception de la fonction objectif jusqu'aux défis des problèmes combinatoires, les études ont permis d'explorer des méthodes exactes et heuristiques, mettant en évidence l'émergence des méta-heuristiques telles que la recherche tabou , l'optimisation par essaim de particules et l'algorithme génétique. Ces approches s'avèrent essentielles pour trouver des solutions optimales dans des contextes où la formulation d'une fonction objectif explicite peut être complexe voire impossible. En créant un équilibre entre exploration et exploitation, ces techniques offrent des solutions rapides et efficaces, ouvrant ainsi la voie à des progrès significatives dans divers domaines.

Chapitre 3 : Résolution du problème (*Bin packing*) par deux méthodes approchées

Chapitre 3 : Résolution du problème (*Bin packing*) par deux méthodes approchées

3.1. Introduction :

Comme exposé dans le chapitre précédent, différentes approches (méthodes) ont été étudiées pour résoudre le problème du *Bin packing*. Cependant, notre choix s'est orienté vers deux méthodes métaheuristiques : l'algorithme génétique associé à l'heuristique « Best-Fit » et l'algorithme de colonie de fourmis.

Le choix de ces deux méthodes s'explique par leur forte inspiration de la nature biologique, ainsi que par leur capacité à explorer efficacement l'espace de recherche et à s'adapter à des problèmes complexes. De plus, elles démontrent une aptitude à équilibrer l'exploration de nouvelles solutions et l'exploitation des solutions actuelles, ce qui s'avère bénéfique pour des problèmes où la meilleure solution n'est pas évidente.

Dans le cadre de notre travail, nous allons mettre en œuvre ces deux approches, puis comparer les résultats obtenus. Enfin, nous présenterons un exemple concret permettant de manipuler les paramètres d'entrée afin d'obtenir des résultats concrets.

3.2. Codage de la solution :

Le codage de la solution nécessitera la représentation des données du problème sous formes de structures qu'on pourra manipuler facilement et efficacement par la suite, dont on cite :

- a. Liste des objets à placer dans les bacs (chaque objet sera représenté par sa taille)
- b. La capacité d'un bac.
- c. Une base de données comportant des instances de paramètres qui nous seront utiles lors de la phase de l'expérimentation.

Prenons un exemple concret de ces derniers, Soit :

Pour 9 objets, et des bacs de capacités = 10 kg au maximum, nous avons les données suivantes :

Une liste chaînée nommée « objets » comportant des objets de différentes taille : **objets = [2, 5, 8, 1, 3, 7, 4, 6, 9]**.

Un entier représentant la capacité d'un bac : **capacite_bin = 10**

Comme résultat nous obtiendrons une liste de bacs, chaque bac est représenté par une liste contenant les objets placés dans ce dernier. Pour notre exemple supposons la solution suivante : **[[8, 2] , [5] , [6, 4] , [7, 3] , [9, 1]]** (Cela signifie que les objets de dimensions 8 et 2 sont placés

dans le premier bac , l'objet 5 est placé dans le deuxième bac seul , le troisième bac contient les objets de taille 6 et 4 , le quatrième bac comporte le objets de dimensions 7 et 3 , et enfin le dernier bac contient les objets de dimensions 9 et 1).

En dernier, afin d'évaluer la performance de notre algorithme, nous affichons un entier représentant le nombre total de bacs utilisés, dans notre cas par exemple : **Nombre de boîtes utilisées = 5.**

3.3. Description des deux approches :

3.3.1. Algorithme génétique :

La méthode de résolution génétique est une méthode moderne s'inspirant de la théorie de l'évolution introduite par Charles Darwin.

Le principe est simple : on cherche à trouver une solution optimale à un problème, pour ce faire, on engendre une population de solution initiale, on les fait évoluer, selon un mécanisme bien défini, puis on effectue une sélection sur les individus de cette population, de manière à garder les plus proches de l'optimum, et on réitère le schéma suivant jusqu'à avoir une population finale proche de l'optimum selon un critère d'arrêt choisi initialement [11][12].

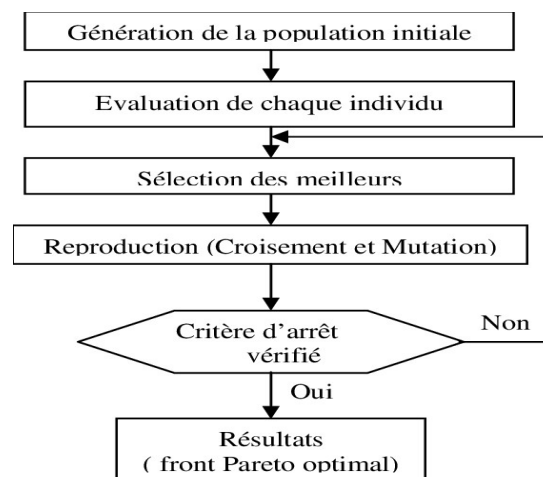


Figure 5 : illustration du processus de l'algorithme génétique pour un nombre d'itération fini

Avant d'entamer l'illustration de cette approche nous devons d'abord définir les composants de cet algorithme :

- **Population :**

C'est une Liste (dont la taille est définie initialement) représentant un ensemble de solutions (disposition des objets dans les bacs) initialisée avec une heuristique qui est « *le Best-fit*

algorithm ». Le choix de ce dernier s'explique par sa performance à retourner les meilleures solutions possibles en un temps restreint, ce qui permet d'être beaucoup plus proche de l'optimum.

- **Individu :**

Élément de la population généré par l'algorithme du Best-fit.

- **Fonction de « fitness » :**

Fonction permettant de mesurer la qualité d'une solution selon un critère choisi, dans notre cas notre fonction retourne le nombre de bacs utilisés (afin de minimiser cette valeur retournée).

- **Sélection :**

Le processus de sélection d'individu peut différer d'un exemple à un autre, nous avons choisis la sélection par roulette, qui attribue des probabilités de sélection à chaque individu en fonction de leur valeur de fitness relative par rapport à la somme totale des valeurs de fitness de la population. Ensuite, un individu est choisi aléatoirement en proportion de ces probabilités, simulant ainsi le tour d'une roulette où des individus avec des fitness plus élevées ont une plus grande chance d'être sélectionnés.

- **Croisement :**

Le croisement simule le processus de recombinaison génétique, où les caractéristiques des parents sont mélangées pour créer une progéniture avec des caractéristiques potentiellement améliorées. Des paires d'individus sélectionnées appelés « parents » sont combinées pour créer de nouveaux individus, souvent appelés "enfants".

Afin de réaliser ce croisement plusieurs approches sont faisables, pour le cas de notre algorithme nous avons choisi la méthode du croisement selon un opérateur appelé *point_crossover*. Ce dernier est choisi de manière aléatoire entre 1 et (la longueur du parent1 – 1) afin de déterminer où le croisement aura lieu. L'enfant est formé en prenant la portion du parent1 jusqu'au point de croisement, puis en ajoutant les objets du parent2 à partir du point de croisement jusqu'à la fin du parent2.

- **Mutation :**

Certaines caractéristiques des nouveaux individus peuvent être modifiées aléatoirement, introduisant ainsi une diversité génétique contribuant à la découverte de solutions potentiellement meilleures.

- **Mise à jour de la population :**

Les nouveaux individus, issus du croisement et de la mutation, remplacent une partie de la population existante. Cette étape permet aux individus les plus performants de rester dans la population.

- **Répétition des étapes :**

Le processus de sélection, croisement et mutation sont répétés sur plusieurs générations. La population évolue au fil du temps, et convergent généralement vers des solutions de plus en plus adaptées à notre problème.

Expliquons maintenant le processus du Best-fit choisi afin d'initialiser une population :

- a. **Tri des objets :**

- Les objets sont triés par ordre décroissant de taille. Ce qui permettra de commencer par placer les objets les plus grands afin de faciliter le remplissage des bacs.

- b. **Initialisation des bacs :**

- Le premier objet trié est placé dans un premier bac. La liste des bacs est initialisée avec ce premier bac contenant un objet.

- c. **Placement des objets restants :**

- Pour chaque objet restant, l'algorithme parcourt les bacs existants pour trouver le "meilleur fit". Ceci est déterminé en cherchant le bac dans lequel l'objet peut être ajouté sans dépasser la capacité du bac. Si aucun bac existant n'est trouvé pour accueillir l'objet, un nouveau bac est créé pour cet objet.

Voici enfin un schéma résumant le principe de notre algorithme génétique :

Algorithme : algorithme génétique

Initialisation d'une population « p » :

Soit **p** la population constituée de $\text{taille}(\mathbf{p})$ de permutations des **n** objets selon l'algorithme du *best-fit*.

Pour $g=1$ jusqu'à $g \leq \text{nombre de générations}$, $g++$ faire {

 Soit **F** la liste des valeurs représentant la qualité de chaque individu

- a. **Sélection :**

 A partir de **F** sélectionner deux individus parents **p1** et **p2** ayant une valeur de **F** élevé par rapport à d'autres individus de la population.

b. Croisement :

Appliquer l'opérateur **point_crossover** aux individus **p1** et **p2** afin de créer deux individus enfants **p12** et **p21**.

c. Mutation :

Selon une constante appelée taux de mutation, choisir aléatoirement si on applique une modification à nos enfants **p12** et **p21**.

d. Mise à jour de la population :

Ajouter les deux enfants créés précédemment **p12** et **p21** à la population initiale. }

Fin pour

Retourner la meilleure solution et l'inverse de son *fitness* qui représente le nombre de bacs utilisés.

3.3.1.1.Choix des paramètres spécifiques à notre algorithme génétique :

Il nous est très utile de fixer les quelques paramètres suivants :

- Taille d'une population
- Nombres de générations (représentant les critères d'arrêt des répétitions).
- Taux de mutation (compris entre 0 et 1).

3.3.2. Algorithme de colonie de fourmis :

Si nous observons une colonie de fourmis à la recherche de nourriture dans les environs du nid, nous nous apercevons qu'elle résout des problèmes tels que celui de la recherche du plus court chemin.

Le comportement de ces insectes a inspiré puis été l'objet d'étude de plusieurs observateurs et chercheurs afin de développer une certaine méthode pouvant résoudre plusieurs problèmes d'optimisation.

En marchant du nid à la source de nourriture et vice-versa (ce qui dans un premier temps se fait essentiellement de façon aléatoire), les fourmis déposent au passage sur le sol une substance odorante appelée phéromones. Cette substance permet ainsi donc de créer une piste chimique,

sur laquelle les fourmis s'y retrouvent. En effet, d'autres fourmis peuvent détecter les phéromones grâce à des capteurs sur leurs antennes.

Ce comportement permet de trouver le chemin le plus court vers la nourriture lorsque les pistes de phéromones sont utilisées par la colonie entière. Autrement dit, lorsque plusieurs chemins marqués sont à la disposition d'une fourmi, cette dernière peut connaître le chemin le plus court vers sa destination.

Retrouver le chemin le plus court fait référence à retrouver la meilleure réponse à la question « quelle est la meilleure solution ? »

Un algorithme de colonies de fourmis est un algorithme itératif à population où tous les individus « fourmis » partagent un savoir commun qui leur permet de guider leurs futurs choix et d'indiquer aux autres individus des directions à suivre ou au contraire à éviter, ce qui veut dire qu'à chaque fois qu'un nouvel individu aura à faire des choix, il pourra s'appuyer sur la connaissance des individus le précédant pour pondérer ses choix.

Explorons maintenant le déroulement de l'algorithme que nous avons développé en nous inspirant de cette approche dans le contexte de la résolution du bin-packing problem :

Tout d'abord, nous définissons un nombre fini de fourmis, représentant le nombre d'itérations de notre algorithme. Ces fourmis vont explorer l'espace de recherche pour trouver la meilleure solution possible, ou du moins une solution proche de l'optimalité. Pour ce faire, nous générons d'abord une solution initiale aléatoire, ou éventuellement via une heuristique. Cette solution représente initialement la meilleure solution trouvée jusqu'à présent. Ensuite, nous évaluons cette solution en utilisant une fonction qui renvoie le nombre de bacs utilisés.

Ensuite, à chaque itération, nous continuons à générer une nouvelle solution et la comparons à la meilleure solution en termes de nombre de bacs utilisés. Si la nouvelle solution est meilleure, nous la considérons comme la meilleure solution actuelle.

Voici l'explication du déroulement de notre algorithme :

Algorithme : algorithme de colonies de fourmis

Initialisation du nombre de bacs de la meilleure-solution avec une valeur infinie

Pour **i** allant de **1** jusqu'à **nombre_de_fourmis**, **i++** {

 #appeler la fonction qui permet de générer une solution aléatoire « **solution** »

#évaluer solution générée à l'aide d'une fonction qui permet de calculer le nombre de bacs de
« **solution** »

Si le nombre de bacs de **solution** < **nombre de bacs de la meilleure solution** alors **meilleure
solution = solution**

Fin Pour

3.3. Conclusion :

Le troisième chapitre a été consacré à la résolution du problème du bin-packing en utilisant deux méthodes approchées différentes : l'algorithme génétique, associé à l'heuristique "Best-Fit", et l'algorithme de colonies de fourmis. Ces choix ont été motivés par leur inspiration biologique et leur capacité à traiter des problèmes complexes nécessitant une exploration efficace de l'espace de recherche.

L'étude a débuté en détaillant le processus de codage de la solution, mettant en lumière la représentation des données du problème et définissant les structures nécessaires pour manipuler les objets, les bacs et les paramètres d'instance. Un exemple concret a été fourni pour illustrer la méthodologie.

Le centre d'attention s'est ensuite déplacé vers la description minutieuse des deux approches. L'algorithme génétique a été exposé en détail, mettant en évidence chaque composant de manière claire, du choix de la population à la mise à jour des individus. Le "Best-Fit" a également été expliqué avec son processus de tri des objets, d'initialisation des bacs, et de placement des objets restants. Tout en soulignant l'importance de la comparaison des résultats obtenus à partir de ses derniers résultats.

Chapitre 4 :

Expérimentation

Chapitre 4 : Expérimentation

4.1. Introduction :

Lors du chapitre précédent, nous avons exploré diverses approches pour aborder cette problématique, en particulier deux métaheuristiques principales combinées à une heuristique simple. Afin de tester les performances de ces deux approches, nous avons développé une interface permettant de générer les résultats souhaités pour chaque méthode, en fonction des données fournies par l'utilisateur. Cette interface offre également la possibilité de comparer les résultats des deux méthodes en termes de nombre de bacs utilisés. Pour obtenir des résultats plus significatifs et améliorés, nous allons explorer quelques benchmarks connus offrant des intervalles d'instances (données d'entrée) déjà prêtes.

4.2. Description des benchmarks :

Les benchmarks jouent un rôle crucial dans l'évaluation des algorithmes de bin packing en leur permettant d'être comparés de manière objective et systématique. Dans cette section, nous détaillerons notre approche pour évaluer les performances des algorithmes de bin packing à travers des benchmarks.

4.2.1. Les benchmarks de la littérature :

4.2.1.1. Falkenauer Benchmark :

Ce sont les instances utilisés par E. Falkenauer dans l'article « *hybrid grouping genetic algorithm for bin packing* ».

Ils sont divisés en deux classes de 80 instances chacune :

- a. "Falkenauer U" :** cette classe a des tailles d'éléments uniformément réparties avec un nombre d'objets (n) entre 120 et 1000, et la capacité des bacs $c = 150$.
- b. "Falkenauer T" :** comprennent ce que l'on appelle les triplets, c'est-à-dire des groupes de trois éléments (un grand, deux petits) qui doivent être affectés au même emplacement dans n'importe quel emballage optimal, avec n compris entre 60 et 501, et $c = 1000$.

4.2.1.2. Scholl Benchmark :

Ce sont les exemples utilisés par A. Scholl, R. Klein et C. Jürgens dans l'affaire « Bison » : une procédure hybride rapide pour résoudre exactement le problème de l'emballage unidimensionnel des bacs. Elles sont divisées en trois ensembles :

- a. Ensemble 'Scholl 1' :** Cet ensemble comprend des instances uniformément distribuées, avec n compris entre 50 et 500, et c varie entre 100 et 150.

- b. Ensemble 'Scholl 2' :** Cet ensemble comprend également des instances uniformément distribuées, avec n compris entre 50 et 500. Cependant, la capacité des bacs est fixée à 1000.
- c. Ensemble 'Scholl 3' :** Cet ensemble comprend des instances uniformément distribuées, avec n compris entre 50 et 500. La capacité des bacs est fixée à 100 000.

Ces variations dans les paramètres comme le nombre d'objets et la capacité des bacs permettent d'explorer la robustesse et l'efficacité des algorithmes dans des conditions diverses [21].

4.2.2. Les benchmarks personnalisés :

Dans notre projet, nous avons également incorporé des benchmarks personnalisés que nous avons ajoutés à notre base de données initiale contenant les benchmarks de la littérature.

4.3. Description de l'interface développée :

4.3.1. Caractéristiques de la machine utilisée (Hardware) :

Processeur (CPU)	Fréquence de l'horloge du processeur	Mémoire vive(RAM)	Type de disque dur	Système d'exploitation
Intel Core i5- 6300U	2,4 GHz	8,00 GO	SSD	Microsoft Windows 10

Figure 6 : Tableau des caractéristiques de la machine utilisée

4.3.2. Langages de programmations et outils utilisés :

Tout d'abord afin de développer nos algorithmes nous avons choisi le langage de programmation python en sa version 3.7.0 avec l'environnement « Pycharm 2018.1 », en raison de sa facilité d'utilisation et de sa richesse en bibliothèques et de manipulation de structures de données tels les listes, les modules etc., permettant de représenter nos solutions efficacement et facilement.



Figure 7 : À droite le logo de l'environnement Pycharm et à gauche le logo de python

Pour mettre cette interface en pratique nous avons choisis de développer une interface web avec HTML, CSS et JAVASCRIPT pour développer l'interface graphique, puis nous avons choisi le Framework « FLASK » ayant pour but d'exécuter à travers le serveur les algorithmes des métaheuristiques choisies et développées en python.

Le choix de la création de l'interface à travers le web s'explique par plusieurs facteurs, dont nous citons :

- **Portabilité et accessibilité** : Contrairement aux applications natives nécessitant une installation préalable, Les applications web peuvent être consultées à partir de n'importe quel navigateur web, sur n'importe quel appareil connecté à Internet à travers une adresse url seulement.
- **Large choix d'outils et de bibliothèques de développement** : Les Framework dont profite le développement web offre une expérience de développement et de consultation par l'utilisateur plus large et riche contrairement aux outils très limités proposés par python.
- **Mises à jour et maintenance** : Les applications web peuvent être mises à jour et déployées facilement sur le serveur, sans nécessiter de mise à jour côté client ce qui facilite la correction des bugs.



Figure 8 : À gauche de logo d'HTML et à droite le logo du CSS



Figure 9 : À gauche le logo de JavaScript et a droite le logo du Framework Flask

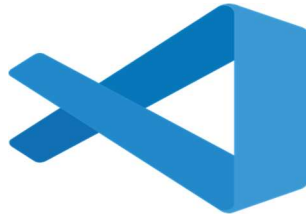


Figure 10 : Logo de l'environnement de développement de l'application « VISUAL CODE STUDIO »

De plus, et afin de générer des graphes comparatifs illustrants les résultats obtenus, nous avons utilisé la bibliothèque « **Matplotlib** » étant l'une des bibliothèques les plus utilisées pour la visualisation de données en Python. Elle offre une grande flexibilité pour créer une variété de graphiques, notamment des graphiques linéaires, des histogrammes, des nuages de points, des graphiques à barres, etc.

Enfin, nous avons utilisé également la bibliothèque « **NumPy** » qui permet d'écrire un code plus efficace, plus concis et plus rapide pour implémenter des algorithmes évolutionnaires comme ceux utilisés lors de la réalisation de ce projet tels que les algorithmes génétiques et les fourmis. Cela permet également de tirer pleinement parti des capacités de calcul de Python pour résoudre des problèmes complexes de manière efficace et robuste.

4.4. Expérimentation et analyse :

4.4.1. Résultat expérimentaux :

Afin de pouvoir comparer les performances des deux approches génétiques et colonie de fourmis, nous présentons ci-dessous les résultats numériques obtenus après l'exécution des deux métaheuristiques. Nous avons testé nos algorithmes en utilisant des instances (entrées de programmes) divisées en plusieurs sous-groupes selon leur appartenance aux benchmarks de la littérature. On note par :

AG : l'algorithme génétique

ACF : l'algorithme de colonie de fourmis

AGB : Moyenne des résultats obtenus par 19 instances du Benchmark en question [22]

NbBin : nombre de bacs utilisés

Classe	Sous-classe	Nombre d'objets	Capacité bin	NbBin A G	NbBin ACF	NbBin AG B
Falkenauer	Falkenauer « U »	120	150	49	70	51
		250	150	100	143	99
		500	150	201	293	198
		1000	150	403	509	400
	Falkenauer « T »	60	100	23	28	20
		120	100	45	58	44
		249	100	94	121	83
		501	100	190	257	167
Scholl	Scholl 1	50	120	25	32	24
		500	120	61	79	59
	Scholl 2	50	1000	28	35	28
		500	1000	268	349	261
	Scholl 3	50	100000	31	38	29
		500	100000	365	405	378

Figure 11 : Tableau comparatifs des résultats obtenus des deux algorithmes et des résultats des benchmarks théoriques cités

4.4.2. Discussion des résultats :

Selon les données du tableau précédent, nous constatons que les résultats de notre algorithme génétique ne s'éloignent pas beaucoup de la moyenne des résultats des benchmarks mentionnés. Il est important de noter que durant chaque exécution, les résultats peuvent varier de 1 à 5 bacs, ce qui explique la divergence entre les deux ensembles de résultats. Et afin de mieux comprendre les différences entre l'approche génétique et celle basée sur les colonies de fourmis, nous avons choisi de créer des graphiques comparatifs de leurs performances respectives.

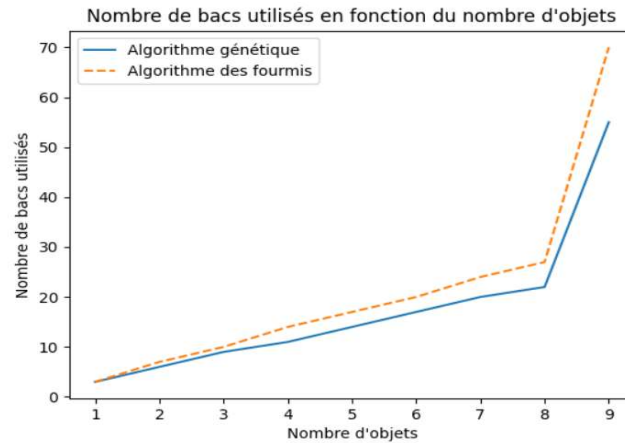


Figure 12 : Graphe comparatifs illustrant le nombre de bacs utilisés pour chaque algorithme en fonction du nombre d'objets

D'après le graphique présenté dans la figure précédente, on constate initialement une différence minime entre les résultats des deux approches lorsque le nombre d'objets est bas, par exemple 10 objets (instance numéro 1), où les deux algorithmes utilisent chacun 4 bacs. Cependant, à mesure que le nombre d'objets augmente, une tendance se dessine : l'algorithme génétique utilise de manière constante un nombre de bacs inférieur à celui de l'algorithme de colonie de fourmis pour un nombre donné d'objets. Par exemple, avec 40 objets, l'algorithme génétique utilise 22 bacs, tandis que l'algorithme de colonie de fourmis en utilise 28 bacs.

Cette différence remarquable dès l'augmentation du nombre d'objets à classer peut être due à plusieurs facteurs, dont nous allons citer certains :

Contrairement à l'algorithme génétique, l'algorithme de colonie de fourmis est connu pour ne pas pouvoir s'adapter à tous types de problèmes d'optimisation, par exemple on peut trouver que sa performance est remarquable lorsqu'il s'agit des problèmes de cheminement et de parcours comme le problème du voyageur de commerce. De plus, la diversité dans l'algorithme de colonie de fourmis peut être limitée par la tendance des fourmis à faire des choix probabilistes en suivant les pistes de phéromones prédominantes, ce qui peut entraîner une convergence prématurée vers des solutions locales. Contrairement, pour l'algorithme génétique, où la diversité est maintenue en favorisant la préservation des individus les plus performants, tout en permettant à des individus moins performants mais différents, d'avoir une chance de se reproduire grâce au processus de sélection. De plus la mutation permet d'introduire des variations aléatoires dans la population, ce qui permet d'explorer de nouvelles régions de l'espace de recherche et d'éviter la convergence prématurée vers des solutions sous-optimales.

Afin d'explorer un peu plus la cause de cette différence dans les performances des deux approches, nous avons opté pour la génération d'un graphique comparatif entre le temps de convergence de chaque métaheuristique :

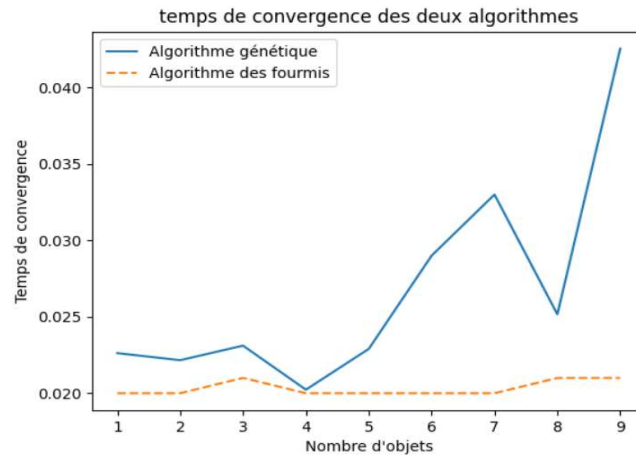


Figure 13 : Graphe comparatif illustrant le temps de convergence des deux algorithmes en fonction du nombre d'objets à classer

Dans le graphique précédent, une disparité frappante apparaît dans le temps nécessaire à la convergence des deux algorithmes. Bien que l'algorithme de colonie de fourmis ne puisse pas rivaliser avec l'algorithme génétique en termes de qualité des résultats produits, il se distingue par sa capacité à converger rapidement vers une solution acceptable. Cette aptitude découle de son efficacité à exploiter les informations locales et à ajuster rapidement ses choix en fonction des pistes de phéromones laissées par les fourmis. En revanche, l'algorithme génétique peut exiger un temps plus considérable en raison de sa nature probabiliste et du besoin de générer et d'évaluer plusieurs générations de solutions potentielles avant d'atteindre une solution satisfaisante.

Par ailleurs, il est observé que lorsque le nombre d'objets à classer dans les bins augmente, l'algorithme de colonie de fourmis maintient un temps de convergence relativement stable. Ce phénomène peut être attribué à la capacité des fourmis à ajuster dynamiquement leurs choix en fonction des informations locales et de la qualité des solutions rencontrées. Cette capacité d'adaptation leur permet de faire face à des environnements changeants et à des problèmes de différentes tailles sans nécessiter de modifications majeures de l'algorithme lui-même.

Pour améliorer les performances de l'algorithme de colonie de fourmis et pour pouvoir comparer équitablement les temps de convergence des deux approches, il est proposé de

spécifier à l'algorithme des fourmis un critère d'arrêt consistant à obtenir le même résultat que celui de l'algorithme génétique pour les mêmes instances d'entrée.

4.5. Conclusion :

Dans ce chapitre, nous avons examiné les deux métaheuristiques choisies, à savoir l'algorithme génétique et l'algorithme de colonie de fourmis, pour résoudre le problème du bin packing. À l'aide de benchmarks de la littérature et de benchmarks personnalisés, nous avons évalué leur efficacité dans l'optimisation de l'utilisation des bacs en fonction du nombre d'objets à ranger.

Les résultats ont révélé que bien que l'algorithme génétique produise des résultats de qualité similaire à la moyenne des benchmarks, il nécessite plus de temps pour converger en raison de sa nature probabiliste et du processus d'évaluation de plusieurs générations de solutions. En revanche, l'algorithme de colonie de fourmis parvient rapidement à des solutions acceptables en exploitant efficacement les informations locales et en ajustant ses choix en fonction des pistes de phéromones laissées par les fourmis. Enfin nous avons proposé une solution afin d'améliorer les performances de l'algorithme de colonie de fourmis et permettre une comparaison plus juste des temps de convergence.

Conclusion générale

Le projet étant réalisé, nous dressons ici le bilan des réalisations accomplies durant notre travail pour atteindre les objectifs fixés par le cahier des charges.

Dans la phase d'état de l'art, nous avons exposé le problème du bin packing, classé parmi les problèmes NP-difficiles, ainsi que ses contraintes, tout en évoquant les travaux antérieurs dans ce domaine. Notre travail s'intéresse, particulièrement, aux méthodes approchées pour résoudre ce problème complexe.

Après avoir exploré diverses approches de résolution du bin packing, nous avons sélectionné deux métaheuristiques : l'algorithme génétique combiné à l'heuristique du Best-fit et l'algorithme des colonies de fourmis. L'algorithme génétique, inspiré de la théorie de l'évolution, génère et améliore des solutions potentielles de manière itérative en créant une population initiale aléatoire, évaluant les individus selon une fonction objectif, puis sélectionnant les plus performants pour la reproduction. Cette reproduction implique le croisement et la mutation pour générer une nouvelle génération, et le processus est répété jusqu'à l'atteinte d'un critère d'arrêt. L'algorithme des colonies de fourmis, quant à lui, s'inspire du comportement des fourmis pour explorer de manière collaborative l'espace de recherche. Chaque fourmi construit une solution en suivant des règles simples basées sur des informations locales et des phéromones déposées par d'autres fourmis. Ce processus itératif d'exploration et d'exploitation permet aux fourmis de converger vers des solutions acceptables.

Pour comparer les performances des deux approches, nous avons testé plusieurs instances et comparé les résultats avec ceux des benchmarks antérieurs. L'analyse a révélé que l'algorithme génétique surpassait celui des colonies de fourmis en termes de robustesse, de diversité et d'exploration de l'espace de recherche.

Enfin, dans le but de comparer les deux approches en termes de temps de convergence et d'obtenir des résultats fiables, nous avons proposé de spécifier un critère d'arrêt pour l'algorithme des colonies de fourmis, basé sur les résultats de l'algorithme génétique pour les mêmes instances d'entrée. Ce critère d'arrêt permettra une comparaison équitable des deux approches et facilitera l'obtention de résultats cohérents.

Bibliographie

- [1] Coffman, E. G., Leung, J. Y.-., & Ting, D. W. (1978). *Bin packing*: Maximizing the number of pieces packed. *Acta Informatica*, 9(3), 263–271, Consulté le : 21 novembre 2023. <https://doi.org/10.1007/bf00288885>
- [2] Mostepha, R. (2008). Résolution de problèmes d'optimisation combinatoire par systèmes artificiels auto-organisés. Thèse de magister, Université Mentouri de Constantine.
- [3] Bayart, F.(n.d.). *Dictionnaire de mathématiques*, Consulté le 25 décembre 2023. <https://www.bibmath.net/dico/index.php?action=affiche&quoi=./c/complexite.html>.
- [4] Index of /ENSEIGNEMENT/projet2/supports/Bin-Packing. (2010, 19 janvier), Consulté le 3 janvier 2024. <https://dept-info.labri.fr/ENSEIGNEMENT/projet2/supports/Bin-Packing/>
- [5] Laurent, A., & Klement, N. (2019). *Bin packing* Problem with priorities and incompatibilities using PSO: application in a health care community. *IFAC-PapersOnLine*, 52(13), 2596–2601, Consulté le 27 février 2024. <https://doi.org/10.1016/j.ifacol.2019.11.598>
- [6] Kampke, T. (1988). Simulated annealing: Use of a new tool in *Bin packing*. *Annals of Operation Research/Annals of Operations Research*, 16(1), 327–332, Consulté le 27 février 2024. <https://doi.org/10.1007/bf02283751>
- [7] Borgulya, I. (2020). A hybrid evolutionary algorithm for the offline *Bin packing* Problem. *Central European Journal of Operations Research*, 29(2), 425–445, Consulté le 27 février 2024. <https://doi.org/10.1007/s10100-020-00695-5>
- [8] Kammarti, R., Ayachi, I., Ksouri, M., & Borne, P. (2013, June 3). Evolutionary approach for the containers Bin-Packing problem. *arXiv.org*, Consulté le 27 février 2024. <https://arxiv.org/abs/1306.0442>
- [9] KHERICI, N. (2020/2021). Optimisation Combinatoire, M1 GADM. Chapitre 1 : Optimisation combinatoire [Matériel d'enseignement]. Université d'Annaba, Consulté le 30 mars 2024. <https://elearning-facsci.univ-annaba.dz/course/view.php?id=147>
- [10] KHERICI, N. (2020/2021). Optimisation Combinatoire, M1 GADM. Chapitre 3 : Optimisation combinatoire [Matériel d'enseignement]. Université d'Annaba, Consulté le 30 mars 2024. <https://elearning-facsci.univ-annaba.dz/course/view.php?id=147>
- [11] EURODECISION. (n.d.). Heuristique métaheuristiques, Consulté le 27 mars 2024. Récupéré sur <https://www.eurodecision.com/algorithmes/recherche-operationnelle-optimisation/heuristiques-meta-heuristiques>

- [12] Duflo, M. (1996). *Algorithmes stochastiques* (1st ed.). Mathématiques et Applications. Springer Berlin, Heidelberg, Consulté le 27 Avril 2024. <https://doi.org/10.1007/978-3-662-03255-3>
- [13] Ghoumari, A. (2018, 10 décembre). Métaheuristiques adaptatives d'optimisation continue basées sur des méthodes d'apprentissage, Consulté le 27 Avril 2024. <https://theses.hal.science/tel-02085935v1>
- [14] Hertz, A. (n.d.). Alain Hertz. <https://www.gerad.ca/~alainh/Optimisation-MTH6311.htm>
- [15] Andre, D. (2024, 1er janvier). Qu'est-ce que l'Optimisation par Essaim de Particules? - *All About AI*, Consulté le 27 Avril 2024. <https://www.allaboutai.com/fr-fr/glossaire-ai/optimisation-par-essaim-de-particules/>
- [16] Besson, J. (2020, 19 mars). Rapport sur un TIPE portant sur Le traitement du problème du Bin-Packing par algorithme génétique (Chapitre 3), Consulté le 30 Avril 2024. <https://perso.eleves.ens-rennes.fr/people/jules.besson/documents.html>
- [17] Wikipedia contributors. (2024, 28 mars). Genetic algorithm. Wikipedia, Consulté le 30 Avril 2024. Récupéré sur https://en.wikipedia.org/wiki/Genetic_algorithm
- [18] Costanzo, A., Luong, T. V., & Marill, G. (2006, 19 mai). Optimisation par colonies de fourmis, Consulté le 30 Avril 2024. https://webusers.i3s.unice.fr/~crescenz/publications/travaux_etude/colonies_fourmis-200605-rapport.pdf
- [19] Vazirani, V. V. (2006). *Algorithmes d'approximation* (N. Schabanel, Trad.). Springer-Verlag France, Consulté le 27 Avril 2024. <https://pdfdrive.to/dl/algorithmes-dapproximation-0>
- [20] GeeksforGeeks. (2023, September 13). Program for Best Fit algorithm in Memory Management. *GeeksforGeeks*, Consulté le 27 Avril 2024. Récupéré sur <https://www.geeksforgeeks.org/program-best-fit-algorithm-memory-management/>
- [21] Delorme, M., Iori, M., & Martello, S. (2018). BPPLIB: A library for *Bin packing* and cutting stock problems. *Optimization Letters*, 12(2), 235-250, Consulté le 03 Mai 2024. <https://site.unibo.it/operations-research/en/research/bpplib-a-bin-packing-problem-library>
- [22] Beasley, J. E. (n.d.). *Bin packing* - one-dimensional, Consulté le 04 Mai 2024. <https://people.brunel.ac.uk/~mastjjb/jeb/orlib/binpackinfo.html>

Annexe

Présentation de l'application :

- **Page d'accueil :**

Lorsqu'on lance notre application, une page d'accueil apparaîtra, offrant un bouton « Commencer » permettant d'accéder à la page de notre application.

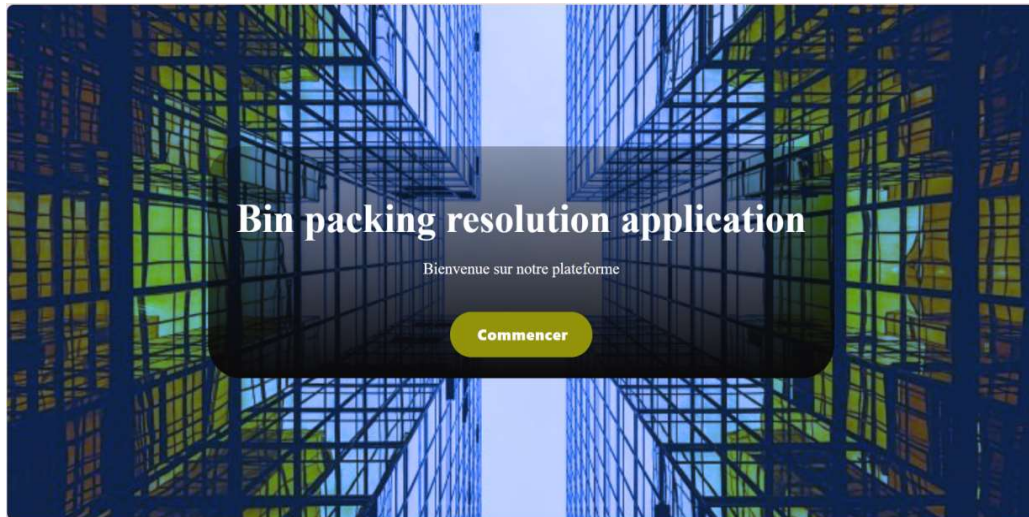


Figure 14 : Page d'accueil de l'application

- **Page de notre application :**

- **Section 1 :** Formulaire contenant les données à fournir (nombres d'objets à ranger, capacité d'un bac, et dimensions des objets).

Figure 15 : Formulaire des données d'entrée

Nous avons offert à l'utilisateur la possibilité d'extraire une instance existante à partir d'une base de données contenant les benchmarks personnalisés et les benchmarks de la littérature.

1 : bouton permettant d'ouvrir la base de données pour extraire des instances

2 : bouton permettant d'enregistrer les données entrées dans la base de données

Figure 16 : Formulaire des différentes données nécessaires pour l'exécution d'un algorithme

Lorsque vous appuyez sur le bouton rouge, une base de données contenant deux tables illustrées dans les deux figures suivantes sera affichée :

Liste des benchmarks de la littérature				
ID	Nom du benchmark	Nombre d'objets	Capacité des bacs	Dimensions des objets
1	Falkenauer U	120	150	[42, 69, 67, 57, 93, 90, 38, 36, 45, 42, 33, 79, 27, 57, 44, 84, 86, 92, 46, 38, 85, 33, 82, 73, 49, 70, 59, 23, 57, 72, 74, 69, 33, 42, 28, 46, 30, 64, 29, 74, 41, 49, 55, 98, 80, 32, 25, 38, 82, 30, 35, 39, 57, 84, 62, 50, 55, 27, 30, 36, 20, 78, 47, 26, 45, 41, 58, 98, 91, 96, 73, 84, 37, 93, 91, 43, 73, 85, 81, 79, 71, 80, 76, 83, 41, 78, 70, 23, 42, 87, 43, 84, 60, 55, 49, 78, 73, 62, 36, 44, 94, 69, 32, 96, 70, 84, 58, 78, 25, 80, 58, 66, 83, 24, 98, 60, 42, 43, 43, 39]
2	Falkenauer U	250	150	[42, 69, 67, 57, 93, 90, 38, 36, 45, 42, 33, 79, 27, 57, 44, 84, 86, 92, 46, 38, 85, 33, 82, 73, 49, 70, 59, 23, 57, 72, 74, 69, 33, 42, 28, 46, 30, 64, 29, 74, 41, 49, 55, 98, 80, 32, 25, 38, 82, 30, 35, 39, 57, 84, 62, 50, 55, 27, 30, 36, 20, 78, 47, 26, 45, 41, 58, 98, 91, 96, 73, 84, 37, 93, 91, 43, 73, 85, 81, 79, 71, 80, 76, 83, 41, 78, 70, 23, 42, 87, 43, 84, 60, 55, 49, 78, 73, 62, 36, 44, 94, 69, 32, 96, 70, 84, 58, 78, 25, 80, 58, 66, 83, 24, 98, 60, 42, 43, 43, 39, 97, 57, 81, 62, 75, 81, 23, 43, 50, 38, 60, 58, 70, 88, 36, 90, 37, 45, 45, 39, 44, 53, 70, 24, 82, 81, 47, 97, 35, 65, 74, 68, 49, 55, 52, 94, 95, 29, 99, 20, 22, 25, 49, 46, 98, 59, 98, 60, 23, 72, 33, 98, 80, 95, 78, 57, 67, 53, 47, 53, 36, 38, 92, 30, 80, 32, 97, 39, 80, 72, 55, 41, 60, 67, 53, 65, 95, 20, 66, 78, 98, 47, 100, 85, 53, 53, 67, 27, 22, 61, 43, 52, 76, 64, 61, 29, 30, 46, 79, 66, 27, 79, 98, 90, 22, 75, 57, 67, 36, 70, 99, 48, 43, 45, 71, 100, 88, 48, 27, 39, 38, 100, 60, 42, 20, 69, 24, 23, 92, 32]

Figure 17 : Table des benchmarks théoriques enregistrés dans la base de données.

Liste des benchmarks personnalisés				
Nom du benchmark	Nombre d'objets	Capacité des bacs	Dimensions des objets	
B5	5	10	[7, 3, 5, 9, 2]	Sélectionner Supprimer
B10	10	10	[1, 6, 4, 8, 9, 2, 3, 7, 5, 10]	Sélectionner Supprimer
B15	15	10	[5, 9, 3, 7, 8, 2, 4, 1, 6, 10, 5, 7, 2, 9, 3]	Sélectionner Supprimer
B20	20	10	[3, 6, 8, 4, 2, 9, 1, 7, 5, 10, 2, 8, 3, 6, 9, 1, 4, 7, 5, 10]	Sélectionner Supprimer
B25	25	10	[9, 2, 5, 7, 1, 8, 3, 6, 10, 4, 5, 3, 8, 9, 7, 1, 2, 6, 10, 4, 3, 5, 9, 8, 2]	Sélectionner Supprimer
B30	30	10	[6, 3, 9, 7, 1, 8, 4, 5, 10, 2, 5, 7, 9, 2, 8, 4, 6, 1, 10, 3, 5, 7, 9, 4, 8, 2, 6, 1, 10, 3]	Sélectionner Supprimer
B35	35	10	[7, 2, 8, 6, 4, 10, 1, 5, 9, 3, 8, 6, 2, 4, 10, 1, 7, 5, 9, 3, 8, 6, 2, 4, 10, 1, 5, 9, 3, 7, 8, 6, 2, 4, 10]	Sélectionner Supprimer
B40	40	10	[3, 8, 4, 7, 1, 10, 2, 6, 9, 5, 4, 8, 7, 1, 10, 2, 6, 9, 5, 3, 8, 4, 7, 1, 10, 2, 6, 9, 5, 3, 8, 4, 7, 1, 10, 2, 6, 9, 5, 3, 8, 4, 7, 1, 10, 2, 6, 9, 5]	Sélectionner Supprimer

Figure 18 : Table des benchmarks personnalisés

Nom du benchmark	Nombre d'objets	Capacité des bacs	Dimensions des objets	1	2
B5	5	10	[7, 3, 5, 9, 2]	Sélectionner	Supprimer

- 2- Bouton permettant de sélectionner un benchmark
- 3- Bouton permettant de supprimer le benchmark

Figure 19 : Une instance de la table des benchmarks personnalisés

Après avoir sélectionné un benchmark, ce dernier sera affiché dans le formulaire illustré dans la figure qui suit :

id du benchmark:

None

Nombre d'objets:

22

Capacité d'un bac:

10

Les dimensions des objets sous forme de liste:

[3, 6, 8, 4, 2, 9, 1, 7, 5, 10, 2, 8, 3, 6, 9, 1, 4, 7, 5, 10]

1

2

Modifier le benchmark

Valider et retourner à l'accueil

- 1- Bouton permettant de modifier le Benchmark
- 2- Bouton permettant d'extraire le Benchmark et revenir à la page principale

Figure 20 : Formulaire du benchmark sélectionné

- **Section 2** : choix entre les deux métaheuristiques (génétique ou colonie de fourmis).

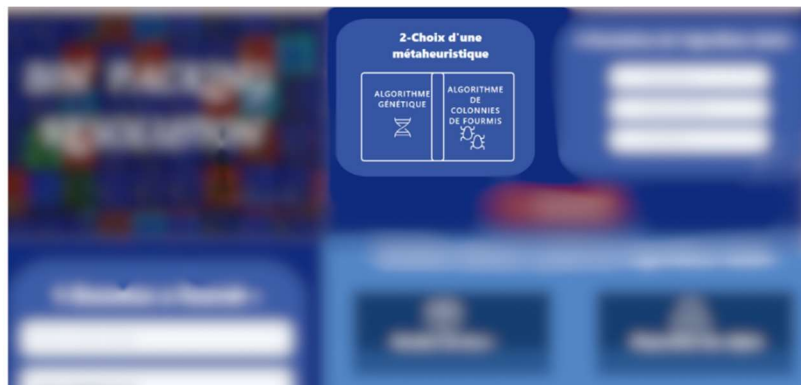


Figure 21 : Section du choix entre les deux métaheuristiques

- **Section 3** : Affichage du formulaire permettant d'entrer les paramètres spécifiques à la métaheuristique choisie.

Dans le cas du choix de l'algorithme génétique, les paramètres demandés sont : la taille d'une population, le nombre de générations et le taux de mutation.

Figure 22 : Les paramètres de l'algorithme génétique

Tandis que si le choix se penche vers l'algorithme de colonie de fourmis, le seul paramètre demandé est le nombre de fourmis.

Figure 23 : Les paramètres de l'algorithme de colonies de fourmis

- **Section 4** : Affichage des résultats après avoir appuyé sur le bouton « Commencer ».

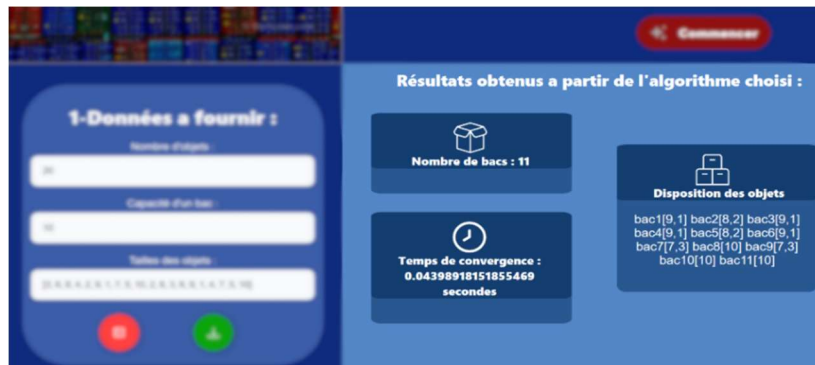


Figure 24 : Les résultats de l'algorithme

- **Section 5** : Génération de deux graphes comparatifs entre les résultats des deux approches, le premier en termes de temps de convergence et le deuxième en termes de nombre de bacs utilisés.



Figure 25 : Boutons générants du graphe